

Chapitre 6 : Entrées, sorties et effets de bord

I Rappel : le type `unit`

Jusqu'ici, toutes les fonctions étudiées calculaient une valeur à partir de paramètres sans modifier l'état global du système (fonctions pures). Pour gérer les affichages ou l'écriture dans des fichiers, nous introduisons la notion d'**effet de bord**.

Définition : Le type `unit` est un type primitif qui ne contient qu'une unique valeur, notée `()`. Elle représente l'absence de valeur de retour intéressante. On l'utilise principalement pour typer les fonctions réalisant des actions ou des affichages.

 **Exemple :** L'évaluation de la constante `()` renvoie son propre type :

```
1  ();;  
2  - : unit = ()
```

Le type `unit` sert également à définir des fonctions qui ne prennent aucun argument utile en entrée, agissant alors comme des procédures.

II Les canaux de communication

En OCaml, les interactions avec l'extérieur (clavier, écran, fichiers) passent par des structures appelées **canaux**. Un canal est unidirectionnel : il est soit ouvert en entrée, soit en sortie, mais jamais les deux simultanément.

 **Remarque :** Dans la suite, il peut être intéressant de faire un parallèle avec les flux d'entrée/sortie en C ou en Java, qui sont également des canaux unidirectionnels et qui fonctionnent de manière similaire.

A Canaux standards (Système UNIX)

Tout processus dispose par défaut de trois canaux de communication fondamentaux :

- `stdin` (type `in_channel`) : Entrée normale du processus, généralement associée au clavier.
- `stdout` (type `out_channel`) : Sortie standard pour les affichages courants, associée à l'écran.
- `stderr` (type `out_channel`) : Sortie standard dédiée aux messages d'erreur.

```
1  stdin;;  
2  - : in_channel = <abstr>  
3  stdout;;  
4  - : out_channel = <abstr>
```

B Ouverture et fermeture de canaux de fichiers

Pour interagir avec des fichiers locaux, on utilise des fonctions d'ouverture qui créent dynamiquement de nouveaux canaux, qu'il convient de fermer après utilisation afin de libérer les ressources système.

- **Pour la lecture :** `open_in : string -> in_channel` et `close_in : in_channel -> unit`
- **Pour l'écriture :** `open_out : string -> out_channel` et `close_out : out_channel -> unit`

 **Exemple :** Ouverture d'un fichier `res.txt` pour y écrire "Hello, world!" puis lecture du contenu de ce fichier :

```

1 let ecrire_texte () =
2   let canal = open_out "res.txt" in
3   output_string canal "Hello, world!\n";
4   close_out canal (* Pas de ";" ici, c'est la fin de la fonction *)
5
6 let lire_texte () =
7   let canal = open_in "res.txt" in
8   let ligne = input_line canal in
9   close_in canal;
10  ligne (* Pas de ";" ici, c'est la valeur renvoyée par la fonction *)
11
12 (* Pour exécuter le scénario au niveau global dans un fichier : *)
13 let () =
14   ecrire_texte ();
15   let texte = lire_texte () in
16   print_endline texte

```

III Écriture dans un canal : fonctions de sortie

Le module Stdlib fournit des fonctions de base pour écrire sur les canaux de sortie. Par défaut, les fonctions n'incluant pas le canal dans leur nom écrivent sur la sortie standard `stdout`.

A Affichage standard

- `print_string : string -> unit` : Affiche une chaîne de caractères.
- `print_int : int -> unit` : Affiche un entier.
- `print_newline : unit -> unit` : Force un retour à la ligne et vide le tampon d'affichage (*flush*).
- `print_endline : string -> unit` : Équivalent à `print_string` suivi de `print_newline`.

B Écriture générale sur un canal

Pour diriger un flux vers un fichier ou vers la sortie d'erreur `stderr`, on utilise les versions généralisées acceptant le canal destinataire en paramètre :

- `output_string : out_channel -> string -> unit`
- `output_char : out_channel -> char -> unit`

💡 **Exemple** : Écriture de données dans un fichier texte local :

```

1 let ecrire_donnees () =
2   let canal = open_out "resultats.txt" in
3   output_string canal "Ligne 1: Bonjour OCaml!\n";
4   output_string canal "Ligne 2: Fin du fichier.\n";
5   close_out canal;;

```

IV Lecture depuis un canal : fonctions d'entrée

De manière analogue aux sorties, la lecture consomme les données disponibles sur un canal d'entrée.

- `read_line : unit -> string` : Lit une ligne de texte depuis l'entrée standard `stdin` (sans le caractère de saut de ligne `\n`).
- `input_line : in_channel -> string` : Lit une ligne depuis le canal spécifié.

- `input_char : in_channel -> char` : Lit un unique caractère.

✖ **Attention** ✖ Lorsqu'une fonction de lecture atteint la fin d'un fichier (*End Of File*), elle s'interrompt en levant l'exception prédéfinie `End_of_file`. Il est impératif d'intercepter cette exception pour clôturer proprement la lecture.

V Ordre d'évaluation et piège de la récursion

L'introduction d'effets de bord casse l'équivalence transparente des expressions mathématiques. La lecture fait avancer la **tête de lecture** du canal ; ainsi, deux appels successifs à `input_line` ne renverront pas le même résultat.

A Le risque de la récursion décalée : exemple du compteur d'octets

L'ordre d'évaluation des arguments d'un opérateur (comme l'addition `+`) n'est pas spécifié par la spécification officielle d'OCaml (il dépend du compilateur, qui évalue souvent de droite à gauche).

Considérons cette tentative **incorrecte** pour compter les octets d'un fichier :

```
1 (* ATTENTION : Code erroné provoquant un Stack overflow *)
2 let rec count_bytes_bad ci =
3   try
4     String.length (input_line ci) + count_bytes_bad ci
5   with
6     | End_of_file -> 0;;
```

- **Pourquoi cela plante-t-il ?** OCaml évalue d'abord le membre de droite de l'addition : `count_bytes_bad ci`. La fonction entre ainsi immédiatement dans l'appel récursif **avant** d'avoir évalué `input_line ci`.
- **Conséquence** : La tête de lecture n'avance jamais sur le fichier avant l'appel suivant, la fonction boucle indéfiniment sur elle-même et sature la pile d'exécution, provoquant un `Stack_overflow`.

B Version correcte : Séquencement explicite via le `let ... in`

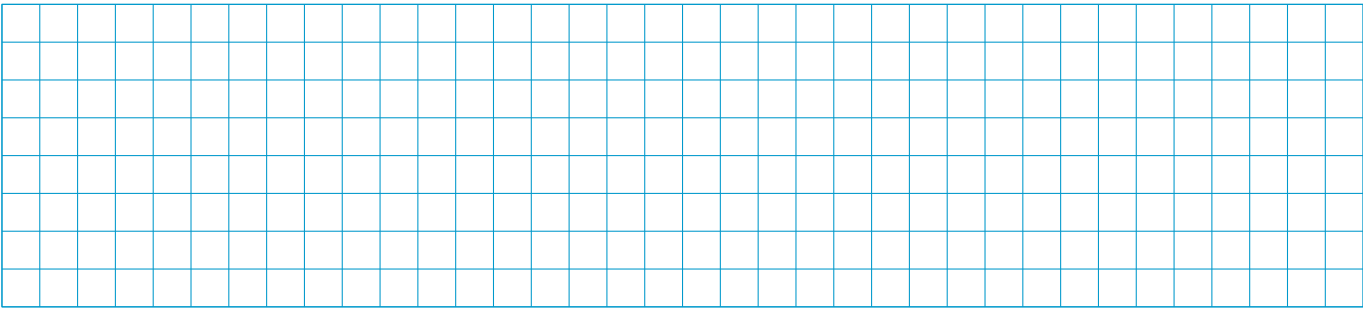
Pour corriger ce bug, il faut forcer l'évaluation de la lecture et l'avancement de la tête de lecture **avant** de déclencher l'appel récursif. On utilise pour cela une liaison locale `let ... in` :

💡 **Exemple** : Implémentation valide et robuste du compteur d'octets :

```
1 let rec count_bytes_correct ci =
2   try
3     let bytes_this_line = String.length (input_line ci) in
4     bytes_this_line + count_bytes_correct ci
5   with
6     | End_of_file -> 0;;
```

Ici, `input_line ci` est impérativement exécuté en premier pour lier sa valeur à `bytes_this_line`, garantissant que la lecture progresse ligne par ligne jusqu'à atteindre la fin du fichier.

🛠 **Application** : écrire une fonction `copier_fichier : string -> string -> unit` prenant en paramètre un fichier source et un fichier destination, qui lit le fichier source ligne par ligne pour le réécrire intégralement dans le fichier destination. On veillera à bien intercepter `End_of_file` et à fermer les deux canaux ouverts.



Contents

I	Rappel : le type <code>unit</code>	1
II	Les canaux de communication	1
A	Canaux standards (Système UNIX)	1
B	Ouverture et fermeture de canaux de fichiers	1
III	Écriture dans un canal : fonctions de sortie	2
A	Affichage standard	2
B	Écriture générale sur un canal	2
IV	Lecture depuis un canal : fonctions d'entrée	2
V	Ordre d'évaluation et piège de la récursion	3
A	Le risque de la récursion décalée : exemple du compteur d'octets	3
B	Version correcte : Séquencement explicite via le <code>let ... in</code>	3